

L4

模型微调与 私有化大模型

8. 低比特微调与量化



1. QAT 量化感知训练
2. QLoRA 量化低秩微调
3. GGUF文件格式
4. 模型蒸馏

方法	方法类别	核心思想	技术特点	适用场景
GPTQ	推理量化（误差优化）	事后补救：Hessian近似最小化输出误差	精度高、逐层优化、计算较慢	GPU服务器部署、离线高精度压缩
AWQ	推理量化（激活感知）	事前保护：基于激活分布调整scale	无需训练、速度快、工程稳定	GPU推理、边缘设备部署（主流方案）
SmoothQuant	激活+权重量化	将激活难度转移到权重（平滑分布）	支持W8A8、硬件友好	高性能推理加速（数据中心）
GGUF	文件格式 / 推理量化	分块量化 + 混合位宽	优化存储与CPU推理效率	CPU、本地部署（如 llama.cpp 生态）
QAT	训练感知量化	在训练中模拟量化误差	精度最高，但训练成本高	工业级高精度模型
QLoRA	微调方法	量化基座 + LoRA低秩训练	显存占用低、训练高效	消费级GPU微调

量化感知训练

量化感知训练 (QAT) 的原理



智泊AI
ZHIPO AI



魔泊云
MoPaaS

概念:

QAT 在训练阶段模拟量化误差, 使模型在训练中 “学习” 低精度计算的特性。

核心机制:

- **前向传播:** 插入量化/反量化操作, 模拟整数化运算带来的离散误差。
- **反向传播:** 采用 STE 梯度不经过量化函数, 而直接传递给上游层。
- **目标:** 让模型在训练过程中 “学会” 容忍量化误差

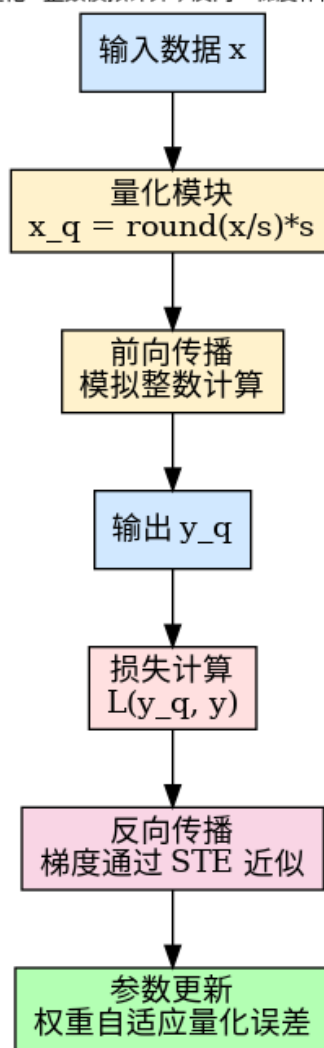
优点:

- 精度最接近 FP16;
- 可训练低比特模型, 例如 (INT4/INT2)
- 适合边缘部署任务, 例如手机、嵌入式设备, 能在低功耗环境中高效运行。

挑战:

- **梯度不稳定:** STE 虽能传播梯度, 但可能引入训练噪声。
- **收敛速度慢:** 量化误差会干扰损失收敛, 需要更长训练时间。
- **高显存消耗:** 因为同时维护浮点权重与量化模拟权重, 内存占用更大。

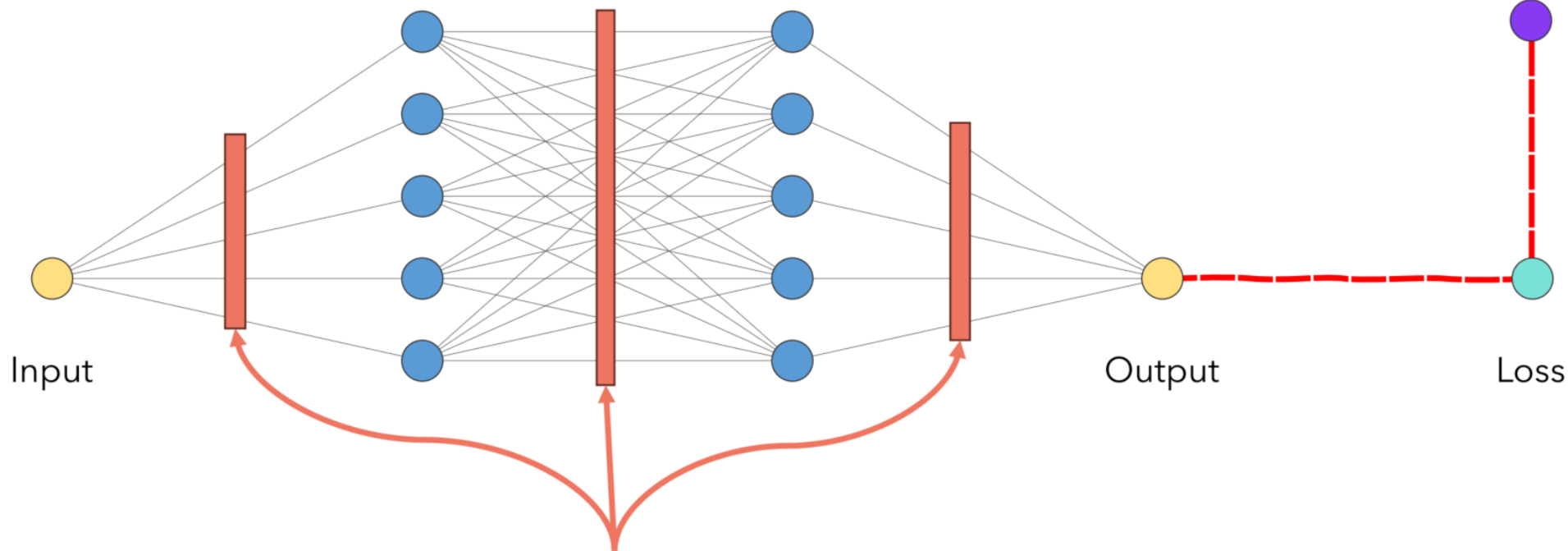
量化感知训练 (QAT) 流程
前向: 伪量化+整数模拟计算; 反向: 梯度补偿与权重更新



Quantization Aware Training (QAT)

We insert some fake modules in the computational graph of the model to simulate the effect of the quantization during training.

This way, the loss function gets used to update weights that constantly suffer from the effect of quantization, and it usually leads to a more robust model.

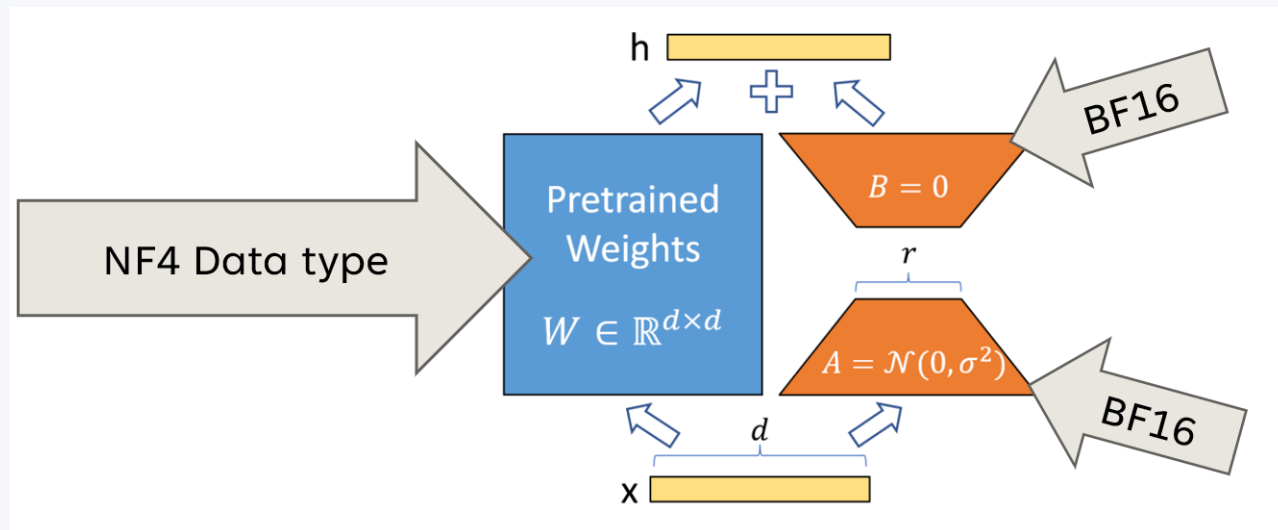


Sequence of **Quantize** and **Dequantize** operations

QLoRA: 量化+低秩微调

核心要点

- **定义**: Quantized Low-Rank Adapter —— 在量化模型上插入可训练低秩适配层。
- **提出背景**: 为在消费级 GPU 上微调百亿级 LLM 而设计。
- **设计目标**:
 - 1. 降低显存占用 (使用 4bit 权重)
 - 2. 保持模型精度 (误差 < 1%)
 - 3. 兼容主流框架 (Transformers + PEFT + bitsandbytes)
- **关键组成**:
 - **NF4 量化**: Normalized Float 4 bit
 - 归一化浮点量化格式;
 - 相比 INT4 能更平滑地保留权重分布。
 - **LoRA 适配层**:
 - 插入可训练的低秩矩阵 A 与 B;
 - 冻结量化主权重, 仅更新 A、B。
 - **双层存储结构**:
 - 主权重 → NF4 (只读)
 - LoRA 参数 → FP16 /BF16 (可训练)



四大机制要点

1. NF4 权重量化 (Normalized Float 4bit)

- 自归一化浮点量化格式;
- 相比 INT4, 更平滑地保留权重分布。

2. 双量化 (Double Quantization)

- 对 scale 参数再次量化;
- 减少约 $0.37\times$ 显存, 不影响反向传播。

3. LoRA 低秩适配层

- 插入矩阵 A、B, 训练参数仅占总量 0.3%;
- 吸收量化误差, 提高稳定性。

4. Paged Optimizer 分页优化

- 动态分配优化器状态;
- 避免大模型训练中的 GPU 内存碎片化。

维度	全精度微调	LoRA	QLoRA
主权重	FP16	FP16	NF4 (4bit)
可训练参数	100%	0.3%	0.3%
显存占用 (13B)	≈96GB	≈48GB	≈24GB
性能损失	-	-0.2%	-0.3%
可运行硬件	8×A100	2×A100	单张 4090 即可

- ⚡ 实际好处
- 显存节省高达 75%
 - 精度几乎无损
 - 可结合 QAT/SmoothQuant 实现端到端压缩
 - 低门槛部署：Hugging Face + PEFT + bitsandbytes 即可上手

- NF4 = Normal Float 4
→ 一种 4-bit 的非线性浮点格式
- 它不是普通的 INT4，而是一种**专门为神经网络权重分布设计的 4bit 浮点编码方式**。
- NF4 的关键目标：
 - **让 4-bit 的数值刻度更符合神经网络权重的真实分布（近似正态分布）。**
 - 因为神经网络权重通常服从： $\mathcal{N}(0, \sigma^2)$
- NF4 利用这种统计特性，让 4bit 也能拟合 FP16 的分布，从而显著提升低比特量化的精度。

- 不像 INT4 那样是均匀整数，NF4 的 16 个值是 **非线性浮点刻度**。

- 示例（不同论文略有差异，但规律一致）：

$[-1.0, -0.66, -0.43, -0.28,$
 $-0.18, -0.11, -0.06, -0.02,$
 $0.02, 0.06, 0.11, 0.18,$
 $0.28, 0.43, 0.66, 1.0]$

- **特点**

- **0 附近更密集** → 这是神经网络权重最密集的区域
- **两端稀疏** → 权重大值本来就少
- 因此 NF4 能更好地“贴合”实际权重分布。

大模型常用数字类型总结表

类别	数字格式	比特	用途	特点
浮点	FP32	32	训练、梯度、优化器	高精度
浮点	FP16	16	AMP训练、推理	较精度、快
浮点	BF16	16	主流训练格式	稳定不溢出
浮点	TF32	19	GPU加速	FP32速度的替代
整数	INT8	8	推理	最普及量化
整数	INT4	4	推理	精度较差
特殊4bit	NF4	4	微调 (QLoRA)	非线性码本，接近 FP16
特殊4bit	FP4	4	Hopper FP4训练	硬件支持
特殊4bit	GPTQ-4bit	4	推理	基于误差补偿
特殊3bit	INT3/Ternary	3	小模型研究	精度差距较大
特殊1bit	Binary	1	二值网络	仅用于研究

GGUF文件格式

为什么要关心 GGUF

一个 7B 模型原始大小约 13GB, 普通笔记本根本跑不动



原始模型

像搬一整个衣柜

13 GB

GGUF 量化后

像装进一个双肩包

4 GB

最终效果

笔记本, 无需显卡

CPU 可跑

GGUF 到底是什么

核心理解: 格式 $\neq$ 算法



智泊 AI
ZHIPU AI



魔泊云
MoPaaS

GGUF 是一种**文件格式**, 不是一种量化算法。

「压缩包」类比

格式 = 包装盒

GGUF 负责「怎么装」

量化 = 压缩方式

Q4_K_M 负责「怎么压」

两者配合

才能得到能跑的小模型

一个 .gguf 文件内含什么

qwen1_5-1_8b-chat-q4_k_m.gguf

模型权重

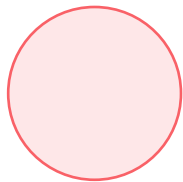
分词器 (Tokenizer)

 **配置信息 (Metadata)**

单文件部署 —— 不需要额外下载 *config.json*、*tokenizer.json* 等文件

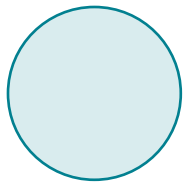
GGUF 的三大优势

为什么它能成为本地部署的事实标准



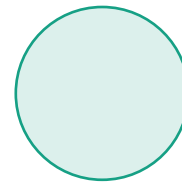
启动快

内存映射(mmap)加载,秒级启动



跨硬件

CPU / GPU / Mac / 树莓派通用



单文件

一个 .gguf 搞定,无需配置

一眼看出差异 —— HuggingFace 格式 vs GGUF

```
# HuggingFace 原始格式
config.json
tokenizer.json
tokenizer_config.json
model-00001-of-00002.safetensors
model-00002-of-00002.safetensors
...
```

```
# GGUF 格式
```

```
qwen1_5-1_8b-chat-q4_k_m.gguf
```

- 就一个文件
- 双击即用

量化等级怎么选

命名规则 + 推荐对照表(以 7B 模型为例)

命名规则



推荐对照表

量化类型	文件大小	质量	推荐场景
Q8_0	~7 GB		追求精度
Q5_K_M	~4.8 GB		平衡之选
Q4_K_M	~4.1 GB		首选推荐
Q3_K_M	~3.3 GB		内存紧张

拿不准

选 Q4_K_M

社区公认性价比最高

注:后缀 _S / _M / _L 分别对应 Small / Medium / Large 三档混合精度策略

怎么生成 GGUF

两步走:HF 模型 → F16 GGUF → 量化 GGUF



1 格式转换:HuggingFace → F16 GGUF (不改变精度)

```
python convert_hf_to_gguf.py ./Qwen1.5-1.8B-Chat  
--outfile qwen-f16.gguf --outtype f16
```

2 真正的量化压缩:F16 → Q4_K_M (降低权重精度)

```
./llama-quantize qwen-f16.gguf qwen-q4_k_m.gguf  
Q4_K_M
```

效果:3.5 GB → 1.1 GB,模型瘦身约 70%

怎么跑起来

三种方案,从最简单到最灵活



方案 1

Ollama

最简单,推荐课堂演示

```
# 拉取并运行(一条命令)
ollama run qwen:1.8b
```

方案 2

llama.cpp

原生工具,最底层

```
# 命令行直接推理
./llama-cli \
  -m qwen-q4_k_m.gguf
  -p "你好"
  -n 128
```

方案 3

Python

对接项目最实用

```
from llama_cpp import Llama

llm = Llama(
    model_path="qwen.gguf"
)
out = llm("你好")
```

课堂彩蛋:我们的 Qwen 客服项目也可导出 GGUF,供同学在自己电脑上演示

模型蒸馏

为什么需要模型蒸馏

大模型能力强但成本高 —— 能否把知识「浓缩」进小模型？



部署贵

GPT-4 级别API费用高昂,延迟达秒级

推理慢

70B+ 大模型在边缘设备完全跑不动

小模型弱

直接训练小模型,能力远不如大模型

教师模型 Teacher

GPT-4 / 70B+

知识
蒸馏 →

暗知识传递

学生模型 Student

Qwen-1.8B / 7B

- 性能逼近大模型
- 体积仅1/10
- 边缘可部署

核心洞见：暗知识 (Dark Knowledge)

大模型输出的概率分布，本身就携带比标签更多的信息



智泊AI
ZHIPO AI



魔泊云
MoPaaS

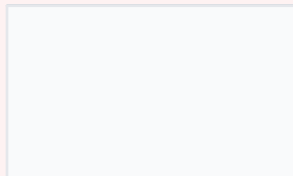
one-hot 只说「是猫」，大模型知道「**80% 猫 / 15% 狐狸 / 5% 狗**」—— 这个相似度梯度就是暗知识

硬标签 vs 软标签

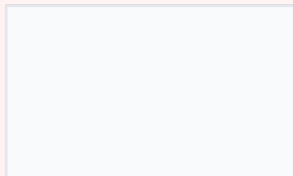
× 硬标签 (one-hot)



猫

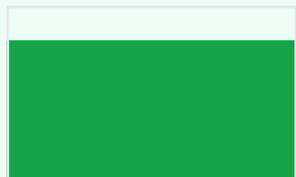


狐狸

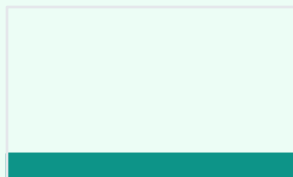


狗

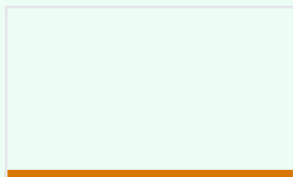
• 软标签 (温度蒸馏)



猫
80%



狐狸
15%



狗
5%

温度系数 T 的作用

默认

$$T = 1$$

普通 Softmax，峰值锐利

推荐

$$T = 4$$

分布变平滑，类间相似度显现

过度

$$T = 10$$

极度平滑，所有类别概率接近

经典蒸馏：流程与损失函数

软标签 + 硬标签联合训练, α 控制两者权重



损失函数组成

$$L = \alpha \cdot L_{\text{soft}}(T) + (1-\alpha) \cdot L_{\text{hard}}$$

L_{soft} (软损失)

学生输出 vs 教师软标签的 KL 散度
学的是「老师怎么想」

温度 T 同时作用于教师和学生

L_{hard} (硬损失)

学生输出 vs 真实 one-hot 标签的交叉熵
保持「不偏离正确答案」

$T = 1$, 与普通训练相同

α 通常取 0.7 ~ 0.9 | T 通常取 3 ~ 10, 具体视任务调参

蒸馏的三大流派

从「只对齐输出」到「对齐中间层」再到「对齐关系结构」



01

响应蒸馏 Response-based

对齐目标

对齐最终输出概率

方法

用教师的 Softmax 输出作软标签训练学生

代表作

Hinton 2015, 经典 KD

优势

最简单, 无需访问内部结构

02

特征蒸馏 Feature-based

对齐目标

对齐中间层隐藏表示

方法

学生不只学答案, 还学「思考过程」; 用线性投影层对齐维度

代表作

FitNets, PKD, TinyBERT

优势

信息更丰富, 适合结构相似的教师-学生对

03

关系蒸馏 Relation-based

对齐目标

对齐样本间的关系结构

方法

「A 和 B 在教师眼中很相似」, 学生也要复现这种相似度几何

代表作

RKD, CRD, SSKD

优势

不依赖架构, 跨模态也适用

LLM 时代的蒸馏 (2023–2025)

从「软标签」到「数据蒸馏」再到「推理能力蒸馏」



黑盒蒸馏

最常见·门槛最低

用 GPT-4 生成大量高质量 Q-A 对，再 SFT 微调小模型。不需要教师 logits。

Alpaca (GPT-3.5 → LLaMA-7B)
Vicuna (ShareGPT → LLaMA)

白盒蒸馏

需要教师 logits

用教师的概率分布做软标签。LLM 生成任务用「反向 KL + on-policy 采样」缓解暴露偏差。

MiniLLM, GKD, DistiLLM

推理蒸馏

2024–2025 最热点

把 o1 / R1 的思维链 (CoT) 输出作为训练数据，让小模型学会「慢思考」推理。

DeepSeek-R1-Distill-Qwen-7B
在数学上逼近 GPT-4o

任务特化蒸馏

工程最实用

把通用大模型蒸馏成领域专家 (客服、医疗、代码)。结合 LoRA 微调效果更好。

你的 L4 Qwen 客服项目
正是这个方向!

蒸馏 vs 量化 vs 剪枝

三种压缩技术对比, 以及如何组合使用



智泊AI
ZHIPO AI



魔泊云
MoPaaS

技术	本质	参数量	需重训练	典型工具	侧部署流
蒸馏	知识迁移	独立小架构	• 需要	SFT / KD-Lib	1 蒸馏 70B → 7B
量化 (GGUF/AWQ)	降低权重精度	不变	× 通常不需要	llama.cpp / AutoAWQ	2 LoRA微调 领域适配
剪枝	删除冗余参数	减少	▲ 通常需微调	SparseGPT / Wanda	3 量化 Q4_K_M
LoRA	低秩适配	增加少量	• 训练新模块	PEFT / LLaMA-Factory	4 GGUF部署 端侧运行

三者不互斥: 蒸馏出 1.8B 小模型 → LoRA 客服微调 → GGUF Q4_K_M 量化 = 完整端侧部署方案

实践：低比特微调与量化

- QAT 感知量化训练
- QLoRA 低比特微调
- 层重要性练习
- UUGF文件格式

请参看练习手册